Table of Contents

Software Architecture & Design Specification

Project: ResiTrack — Apartment / Residencies Organizer **Course:** Software Engineering

---

1. Introduction

Describes the architecture and detailed design of ResiTrack, translating `SRS.docx` requirements into a concrete system structure. References: `SRS.docx`, `TestPlan.docx`.

---

2. Architecture

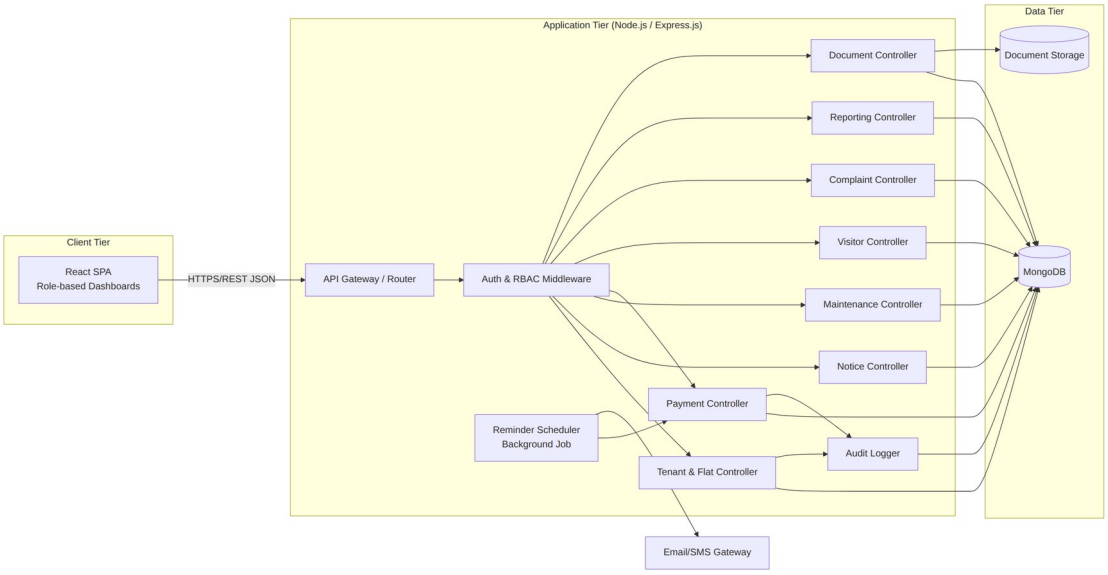
2.1 Architecture Pattern

3-tier client–server, with the server organized as **MVC** (required by NFR-09):

- **Presentation (client):** React SPA — role-specific dashboards, consumes the REST API.
- **Application (server):** Node.js + Express.js — **Models** (Mongoose schemas), **Controllers** (business logic per module), **Routes** (guarded by auth/RBAC middleware).
- **Data:** MongoDB, accessed only through the model layer.

Chosen for clean separation of concerns, a natural mapping onto REST resources, and the team's existing MERN experience.

2.2 Component Diagram, Description & Requirements Traceability



Component Diagram

Component	Responsibility	Requirements
React SPA	Renders UI per role, client-side routing/state	—
API Gateway / Router	Entry point; dispatches to controllers	—
Auth & RBAC Middleware	Verifies token, enforces role access before any controller runs	FR-02, FR-14, SEC-01–04
Tenant & Flat Controller	Tenant registration, flat inventory	FR-01, FR-12
Notice Controller	Notice creation/broadcast	FR-03
Maintenance Controller	Requests, status updates, Kanban view	FR-04, FR-05, FR-09
Payment Controller	Payments, receipts	FR-06, FR-15
Visitor Controller	Visitor entry logging	FR-08
Complaint Controller	Complaint filing/tracking	FR-13
Reporting Controller	Owner financial/occupancy summaries	FR-10
Document Controller	Upload with type/size validation	FR-11, SEC-06
Reminder Scheduler	Background job triggering payment reminders	FR-07
Audit Logger	Immutable log of admin actions	NFR-10, SEC-05

Component	Responsibility	Requirements
MongoDB / Document Storage	Persistent store	supports all above

(All controllers together, under the MVC pattern with ≥60% test coverage, satisfy NFR-09; deployment choices below address NFR-01, NFR-02, NFR-08.)

2.3 Security Architecture

- **Authentication:** Token (JWT/session) issued at login; carries user ID + role.
- **Authorization:** RBAC middleware checks the role claim against each route's allowlist before controller logic runs (FR-14, SEC-03); default is deny.
- **Passwords:** bcrypt, cost ≥12, never logged or returned in responses (SEC-01).
- **Transport:** TLS 1.2+ enforced at the proxy layer; HTTP redirected (SEC-02).
- **Sessions:** 30-minute inactivity timeout (SEC-04).
- **Audit trail:** state-changing admin requests logged append-only — actor, action, timestamp (SEC-05).
- **Input validation:** file-type/size allowlist for uploads (SEC-06); schema validation on all request bodies to mitigate injection.

2.4 Deployment & Technology Stack

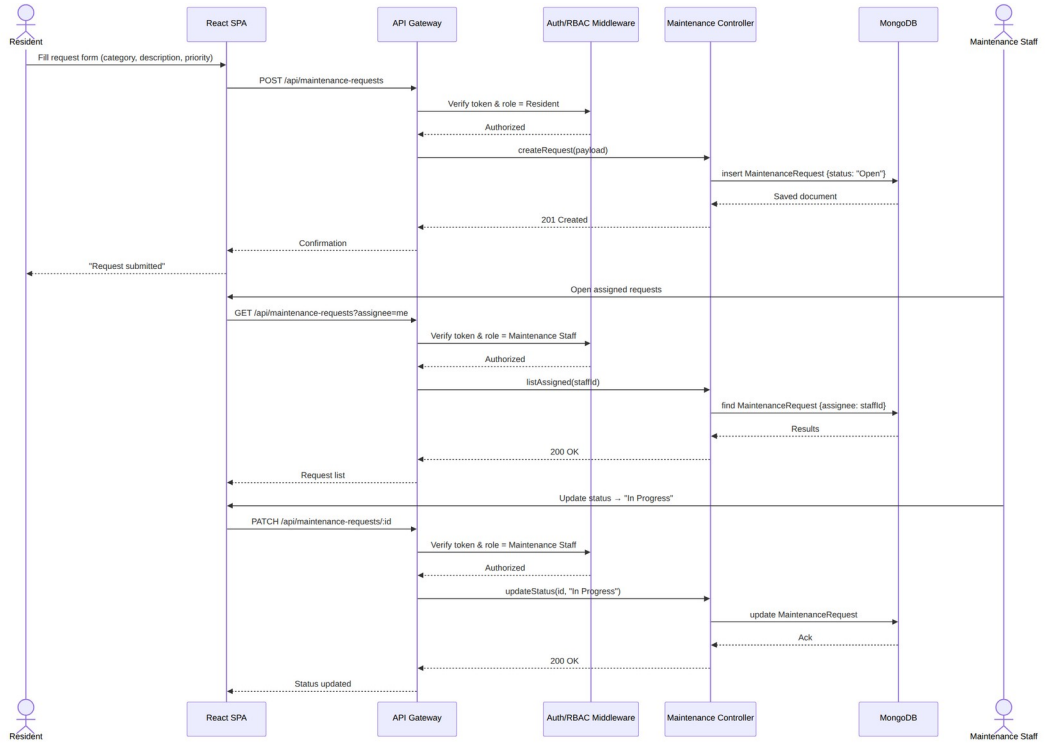
Client (static React build) and server deployed as separate services on free-tier PaaS (Render/Railway); MongoDB Atlas free tier.

Layer	Technology
Frontend	React, React Router, Axios
Backend	Node.js, Express.js
Database	MongoDB, Mongoose
Auth	JWT, bcrypt
Scheduling	node-cron (FR-07 reminders)
PDF generation	pdfkit / puppeteer (FR-15 receipts)
Testing	Jest/Mocha, Supertest, Cypress/Playwright

3. Design

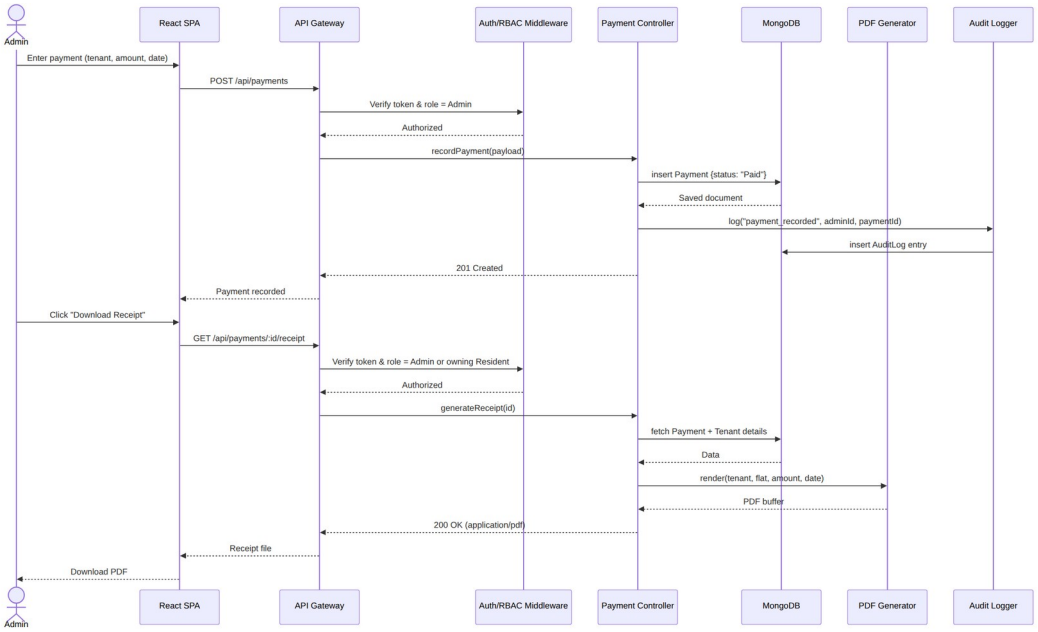
3.1 UML Sequence Diagrams

3.1.1 Raise Maintenance Request → Update Status (UC-04, UC-05)



Sequence Diagram 1

3.1.2 Record Rent Payment → Generate Receipt (UC-06, UC-15)



Sequence Diagram 2

3.2 API Design

All endpoints prefixed `/api`; require a valid token unless marked Public. Every protected route passes through the Auth & RBAC middleware (§2.3) first.

Method	Endpoint	Roles	Description
POST	`/api/auth/login`	Public	Authenticate (FR-02)
POST	`/api/tenants`	Admin	Register tenant (FR-01)
POST	`/api/flats`	Admin	Add flat (FR-12)
PATCH	`/api/flats/:id`	Admin	Edit/deactivate flat (FR-12)
POST	`/api/notices`	Admin	Post notice (FR-03)
GET	`/api/notices`	All	List active notices
POST	`/api/maintenance-requests`	Resident	Raise request (FR-04)
GET	`/api/maintenance-requests`	Admin, Maintenance	List/filter (FR-09)
PATCH	`/api/maintenance-requests/:id`	Maintenance	Update status (FR-05)
POST	`/api/payments`	Admin	Record payment (FR-

Method	Endpoint	Roles	Description
			06)
GET	`/api/payments/:id/receipt`	Admin, owning Resident	PDF receipt (FR-15)
POST	`/api/visitors`	Resident	Log visitor (FR-08)
POST	`/api/complaints`	Resident	File complaint (FR-13)
GET	`/api/reports/financial-summary`	Owner	Financial summary (FR-10)
POST	`/api/documents`	Admin	Upload document (FR-11)
GET	`/api/audit-logs`	Admin	Retrieve audit logs (NFR-10)

3.3 Error Handling

Centralized Express error middleware; consistent JSON envelope: `{ "error": { "code", "message", "field?" } }`.

Scenario	Status	Handling
Invalid/missing request fields	400	Schema validation rejects before controller runs; field-level message
Missing/invalid auth token	401	Auth middleware short-circuits
Valid token, wrong role	403	RBAC middleware short-circuits; no data access (FR-14, SEC-03)
Resource not found	404	Existence check before action; generic message, no data leakage
Disallowed/oversized upload	413 / 415	Rejected before write (SEC-06)
Duplicate resource (e.g., email)	409	Unique-index violation → client-friendly message
Session expired	401 (`SESSION_EXPIRED`)	Client redirects to login (SEC-04)
Unexpected server error	500	Full trace logged server-side only; generic client message

3.4 Data Design

Core MongoDB collections: `users` (with `role`), `tenants`, `flats`, `notices`, `maintenanceRequests`, `payments`, `visitors`, `complaints`, `documents`, `auditLogs`. References (e.g., `tenant.flatId`) use indexed Mongoose `ObjectId`s to support the NFR-08 scalability target.